



HASHDIT

HashDit

Smart Contract Security

ListaDao ListaV3 Core & Periphery Contracts Security Audit Report

ListaDao

27 April 2026 – 30 April 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	27 April 2026 – 30 April 2026
Overall Risk [Optional]	Low
Project Name	ListaDao
Github	https://github.com/lista-dao/lista-v3
Commit	main

Smart Contract List

No.	Contract Name	Verdict	Details
1	Core	See Findings	See Findings
2	Periphery	See Findings	See Findings

Findings

[L01] Missing `_disableInitializers()` in `ListaV3Factory` and `NonfungiblePositionManager` implementations

Contract `ListaV3Factory.sol` & `NonfungiblePositionManager.sol`

Severity **Low**

Description

Description

Neither `ListaV3Factory` nor `NonfungiblePositionManager` invokes `_disableInitializers()` within its constructor.

Although the deployment script attempts to neutralize the implementation slots by calling `factoryImpl.initialize(Oxdead)` and `npmlImpl.initialize(Oxdead)` after deployment, these calls can occur in transactions separate from the implementation deployments themselves if an atomic deployment is not performed.

This can allow a transaction between the deployment and the burn call, invoking `initialize` first and claiming ownership of the implementation contract to point the proxy towards a malicious hidden implementation contract, also known as the CPIMP attack (Cross-Proxy Initialization Malicious Proxy).

Recommendation Consider adding `constructor() { _disableInitializers(); }` to both `ListaV3Factory` and `NonfungiblePositionManager`.

Status Acknowledged, no fix implemented. The deployment script is enforced to burn the implementation by initializing the implementation contract using the `Oxdead` address.

[I01] Removal of `NoDelegateCall` protection in `createPool` shifts trust to the proxy admin

Contract `ListaV3Factory.sol` & `NonfungiblePositionManager.sol`

Severity **Informational**

Description

The `ListaV3Factory.createPool()` function no longer carries the `NoDelegateCall` protection present in the original Uniswap V3 design.

Description

This change is required by the proxy upgrade pattern, since the implementation must be reachable through `delegatecall` from the proxy. As a result, the only remaining safeguard against a malicious or buggy upgrade is the proxy admin and its associated timelock.

If the proxy admin is an externally owned account or otherwise compromised, an attacker controlling the upgrade path can replace the implementation and drain or corrupt all factory state, including pool registrations and protocol configuration.

JavaScript

```
/// @inheritdoc IListaV3Factory
function createPool(
    address tokenA,
    address tokenB,
    uint24 fee
) external override returns (address pool) {
    require(tokenA != tokenB);
    (address token0, address token1) = tokenA < tokenB ? (tokenA,
tokenB) : (tokenB, tokenA);
    require(token0 != address(0));
    int24 tickSpacing = feeAmountTickSpacing[fee];
    require(tickSpacing != 0);
    require(getPool[token0][token1][fee] == address(0));
    pool = deploy(address(this), token0, token1, fee, tickSpacing);
    getPool[token0][token1][fee] = pool;
    // populate mapping in the reverse direction, deliberate choice
    to avoid the cost of comparing addresses
    getPool[token1][token0][fee] = pool;
    emit PoolCreated(token0, token1, fee, tickSpacing, pool);
}
```

Recommendation

Document the proxy upgrade trust model clearly so users understand that factory integrity depends on proxy administration. Configure the proxy admin as a multisig wallet combined with a timelock controller so that any upgrade requires multiple authorized signers and a public delay window during which the community can detect and respond to a malicious upgrade attempt.

Status

Fixed, the ListaV3Factory is now reverted to non-upgradeable with the noDelegateCall modifier restored

[I02] Token descriptor became mutable via proxy upgrade in NonfungiblePositionManager

Contract ListaV3Factory.sol

Severity Informational

Description

Description

In the original Uniswap V3 design, _tokenDescriptor was declared as an immutable variable, meaning the contract that renders position NFT metadata was fixed at construction and could never be altered.

In the Lista version, `_tokenDescriptor` has been converted into a regular storage slot to accommodate the proxy upgrade pattern. There is no dedicated setter function exposed for it, but the proxy admin can still replace the descriptor by performing an implementation upgrade that writes a new value to the slot. As a consequence, the rendered metadata of every previously minted position NFT can be modified after the fact, which weakens the integrity guarantees that holders of these NFTs would otherwise expect, and shifts trust onto the proxy administrator.

JavaScript

```
function initialize(address _tokenDescriptor_) external initializer {
    __ERC721Permit_init('Lista V3 Positions NFT-V1', 'LIS-V3-POS',
        '1');
    _tokenDescriptor = _tokenDescriptor_;
    _nextId = 1;
    _nextPoolId = 1;
}
```

Recommendation	Document this trust assumption clearly so that NFT holders and integrators understand that position metadata rendering is upgrade controlled.
Status	Acknowledged, the timelock contract will retain the permissions to perform upgrades and is considered to be trusted.

[I03] NatSpec references Lista callback names while on chain selectors remain Uniswap

Contract ListaV3Factory.sol

Severity **Informational**

Description	Description The current NatSpec documentation in <code>IListaV3PoolActions.sol</code> references callback hooks using the the original <code>uniswapV3MintCallback</code>, <code>uniswapV3SwapCallback</code>, and <code>uniswapV3FlashCallback</code>. However, for accurate naming purposes, the actual callback selectors invoked by the pool contracts on chain should be readjusted to <code>listaV3MintCallback</code>, <code>listaV3SwapCallback</code>, and <code>listaV3FlashCallback</code>. This is purely a documentation inconsistency and has no runtime effect, but it may mislead integrators who read the interface documentation and implement callback handlers under the wrong function names. Such integrators would find that their callbacks are never invoked, causing failed mints, swaps, or flash loans.
Recommendation	For accurate naming purposes, the actual callback selectors invoked by the pool contracts on chain should be readjusted to <code>listaV3MintCallback</code> , <code>listaV3SwapCallback</code> , and <code>listaV3FlashCallback</code> , along with the callback interfaces being renamed to

	IListaV3SwapCallback.sol, IListaV3MintCallback.sol and IListaV3FlashCallback.sol respectively
Status	Acknowledged, no fix implemented, the uniswap callback names are retained for integration purposes.

[I04] _nameHash and _versionHash moved from immutable to storage in ERC721Permit	
Contract	NonfungiblePositionManager.sol
Severity	Informational
Description	Description In the modified NonfungiblePositionManager, _nameHash and _versionHash were changed from immutable to storage variables. This introduces two concerns. 1. These values now occupy upgrade sensitive storage slots. Since __ERC721Permit_init is gated by the initializer modifier, an upgrade cannot re-run it to update the name or version, and no reinitializer is defined for this purpose. The behavior is acceptable but effectively freezes the name and version permanently after deployment. 2. Both __ERC721Permit_init and its _unchained variant are marked with the initializer modifier. Under OpenZeppelin 4.x, the nested call works because the inner invocation observes _initializing == true and skips the toggle, but the established convention is to use onlyInitializing on the _unchained variant. Although the risk is low given the pinned solc version 0.7.6, a future migration to a stricter OpenZeppelin release could cause this nested initializer pattern to revert. <pre> JavaScript /// @dev The hash of the name used in the permit signature verification bytes32 private _nameHash; /// @dev The hash of the version string used in the permit signature verification bytes32 private _versionHash; </pre>
Recommendation	If future flexibility for the EIP-712 name or version is desired, define a reinitializer guarded function that allows updating _nameHash and _versionHash during an upgrade
Status	Acknowledged, design choice with storage layout safety addressed.

[I05] __nextId and _nextPoolId rely on initialize() to set their non-zero starting values	
Contract	NonfungiblePositionManager.sol

Severity	Informational
Description	Description In the upgradeable NonfungiblePositionManager, _nextId and _nextPoolId no longer carry their inline = 1 initialization. Both counters are now assigned inside initialize() instead. If initialize() is omitted, delayed, or front run on a freshly deployed proxy, both variables remain at 0. A value of 0 collides with the "skips 0" sentinel used by cachePoolKey to detect uncached pools and by mint to assign new token IDs, which causes pool caching and minting logic to misbehave. <pre> JavaScript function initialize(address _tokenDescriptor_) external initializer { __ERC721Permit_init('Lista V3 Positions NFT-V1', 'LIS-V3-POS', '1'); _tokenDescriptor = _tokenDescriptor_; _nextId = 1; _nextPoolId = 1; } </pre>
Recommendation	Ensure that proxy deployment and the initialize() call are performed atomically in the same transaction so the contract is never reachable in an uninitialized state
Status	Acknowledged, the deployment script will atomically initialize the proxy contract to ensure that the respective id variables are initialized

Known UniswapV3 Findings

Note: Only significant findings were highlighted, all other issues should be referenced in the Resources section

[H01] TransferHelper.safeTransfer lacks contract existence check

Contract TransferHelper.sol

Severity **High**

Description

TransferHelper.safeTransfer performs a low level call against the token address and considers the transfer successful when success is true and the returned data is either empty or decodes to true.

JavaScript

```
/// @notice Transfers tokens from msg.sender to a recipient
/// @dev Calls transfer on token contract, errors with TF if
transfer fails
/// @param token The contract address of the token which will be
transferred
/// @param to The recipient of the transfer
/// @param value The value of the transfer
```

```
function safeTransfer(
    address token,
    address to,
    uint256 value
) internal {
    (bool success, bytes memory data) =
```

```
token.call(abi.encodeWithSelector(IERC20Minimal.transfer.selector, to,
value));
    require(success && (data.length == 0 || abi.decode(data,
(bool))), 'TF');
}
```

By EVM design, a low level call to an account with no code returns success with empty return data. If a token contract has been self-destructed, or in any other situation where the token address no longer hosts code, safeTransfer will report success even though no balance changed. Inside swap(), this means the pool can transfer the input token in from the user, attempt to send the output token out, observe a fake success, and leave the user without the output tokens they paid for.

Recommendation	Consider verifying that the token address has non zero code size, for example using <code>extcodesize</code> or an equivalent check, before issuing the low level call, and revert with a clear error if it does not.
Status	Acknowledged inherited uniswapV3 behavior, no fix implemented. It is presumed that only tokens without such edge case behaviors will be accepted into ListaV3 pools.

[M01] Unbounded swap loop enables denial of service through tick spam

Contract ListaV3Pool.sol

Severity **Medium**

Description

ListaV3Pool.swap() iterates while there is remaining input or output and the price has not reached the limit, advancing one initialized tick at a time. The number of iterations is bounded only by gas.

The factory only requires `tickSpacing > 0 && tickSpacing < 16384` when enabling a fee tier, with no minimum tick spacing and no minimum liquidity per position. An attacker, especially a miner or validator on a low fee chain, can create thousands of 1 wei positions across many ticks. Subsequent swaps then have to cross enough ticks to exceed the block gas limit, causing swaps to revert and effectively denying use of the pool.

Description

JavaScript

```
// continue swapping as long as we haven't used the entire
// input/output and haven't reached the price limit
while (state.amountSpecifiedRemaining != 0 && state.sqrtPriceX96
!= sqrtPriceLimitX96) {
    StepComputations memory step;

    step.sqrtPriceStartX96 = state.sqrtPriceX96;

    (step.tickNext, step.initialized) =
    tickBitmap.nextInitializedTickWithinOneWord(
        state.tick,
        tickSpacing,
        zeroForOne
    );

    // ensure that we do not overshoot the min/max tick, as the
    // tick bitmap is not aware of these bounds
    if (step.tickNext < TickMath.MIN_TICK) {
        step.tickNext = TickMath.MIN_TICK;
    } else if (step.tickNext > TickMath.MAX_TICK) {
        step.tickNext = TickMath.MAX_TICK;
    }
}
```

Recommendation	Consider enforcing a meaningful minimum tickSpacing when enabling a fee tier in the factory, and consider also requiring a minimum liquidity per position in the pool so that creating positions is not effectively free. Document the chosen bounds and ensure that across all enabled fee tiers the worst case number of initialized ticks a swap can cross stays well within typical block gas limits.
Status	Acknowledged inherited uniswapV3 behavior, no fix implemented. It is presumed that the owner would set reasonable tick spacings.

[M02] Pool initialize is permissionless and can be front run to set an unfair price

Contract	ListaV3Pool.sol
Severity	Medium
Description	Description ListaV3Pool.initialize() is external with no access control and only requires that <code>slot0.sqrtPriceX96 == 0</code>. Anyone can call it on a freshly deployed pool. An attacker watching the mempool can front-run a deployer's <code>initialize()</code> transaction and set an unfair <code>sqrtPriceX96</code>. When the deployer or a liquidity provider then mints liquidity assuming the intended price, the attacker swaps against the mispriced pool and extracts value from those initial deposits. <pre> JavaScript // continue swapping as long as we haven't used the entire input/output and haven't reached the price limit while (state.amountSpecifiedRemaining != 0 && state.sqrtPriceX96 != sqrtPriceLimitX96) { StepComputations memory step; step.sqrtPriceStartX96 = state.sqrtPriceX96; (step.tickNext, step.initialized) = tickBitmap.nextInitializedTickWithinOneWord(state.tick, tickSpacing, zeroForOne); // ensure that we do not overshoot the min/max tick, as the tick bitmap is not aware of these bounds if (step.tickNext < TickMath.MIN_TICK) { step.tickNext = TickMath.MIN_TICK; } else if (step.tickNext > TickMath.MAX_TICK) { step.tickNext = TickMath.MAX_TICK; </pre>

```
}
```

Consider moving price initialization into the constructor so that it cannot be observed and front run, or restrict initialize() to a privileged deployer such as the factory or the pool creator.

Recommendation

At minimum, document the risk prominently and ensure that any official deployment flow performs deployment and initialization in a single transaction, for example through a deployer contract that creates the pool and immediately initializes and mints in the same call.

Status

Acknowledged inherited uniswapV3 behavior, no fix implemented. ListaDao has indicated that all pool creation will follow a create+initializes+seed atomically.

[M03] Swapping on zero liquidity allows arbitrary price manipulation

Contract ListaV3Pool.sol & SwapMath.sol

Severity **Medium**

Description

In SwapMath.computeSwapStep, when liquidity == 0 and a fee is enabled, both amountRemainingLessFee and amountIn evaluate to 0, the condition amountRemainingLessFee >= amountIn is satisfied, and sqrtRatioNextX96 is set directly to sqrtRatioTargetX96. As a result, swapping 1 wei across a tick range with no active liquidity moves the pool's price to whatever target the caller chooses.

This lets anyone set an arbitrary price between initialization and the first liquidity provision, or whenever liquidity providers temporarily withdraw all liquidity. A subsequent liquidity provider depositing at what they assume is the correct price can be sandwiched and drained.

Description

Swapping 1 wei in exactIn with a liquidity of zero and a fee enabled will cause amountRemainingLessFee and amountIn to be zero

As amountRemainingLessFee == amountIn, the next square root ratio will be the square root target ratio:

JavaScript

```
if (exactIn) {
    uint256 amountRemainingLessFee =
FullMath.mulDiv(uint256(amountRemaining), 1e6 - feePips, 1e6);
    amountIn = zeroForOne
```

```

        ? SqrtPriceMath.getAmount0Delta(sqrtRatioTargetX96,
sqrtRatioCurrentX96, liquidity, true)
        : SqrtPriceMath.getAmount1Delta(sqrtRatioCurrentX96,
sqrtRatioTargetX96, liquidity, true);
    if (amountRemainingLessFee >= amountIn) sqrtRatioNextX96 =
sqrtRatioTargetX96;

```

The next square root ratio assignment results in updates to the pool's price and tick:

JavaScript

```

    // shift tick if we reached the next price
    if (state.sqrtPriceX96 == step.sqrtPriceNextX96) {
        // if the tick is initialized, run the tick transition
        if (step.initialized) {
            // check for the placeholder value, which we replace
            // with the actual value the first time the swap
            // crosses an initialized tick
            if (!cache.computedLatestObservation) {
                (cache.tickCumulative,
cache.secondsPerLiquidityCumulativeX128) = observations.observeSingle(
                    cache.blockTimestamp,
                    0,
                    slot0Start.tick,
                    slot0Start.observationIndex,
                    cache.liquidityStart,
                    slot0Start.observationCardinality
                );
                cache.computedLatestObservation = true;
            }
            int128 liquidityNet =
                ticks.cross(
                    step.tickNext,
                    (zeroForOne ? state.feeGrowthGlobalX128 :
feeGrowthGlobal0X128),
                    (zeroForOne ? feeGrowthGlobal1X128 :
state.feeGrowthGlobalX128),
                    cache.secondsPerLiquidityCumulativeX128,
                    cache.tickCumulative,
                    cache.blockTimestamp
                );
            // if we're moving leftward, we interpret
            // liquidityNet as the opposite sign
            // safe because liquidityNet cannot be
            type(int128).min
            if (zeroForOne) liquidityNet = -liquidityNet;

            state.liquidity =
LiquidityMath.addDelta(state.liquidity, liquidityNet);

```

```
}
```

Recommendation

Consider disallowing price changes when liquidity == 0, for example by reverting inswap() when no liquidity is active, or by ensuring that traversing a zero liquidity region cannot move the price without a corresponding token payment.

Alternatively, ensure pools can only be created in flows that atomically deploy, initialize, and seed liquidity, and warn integrators that pools whose liquidity has been fully withdrawn should not be re-entered without re-evaluating the price.

Status

Acknowledged inherited uniswapV3 behavior, no fix implemented.

[L02] Interest bearing tokens can satisfy the mint and flash balance checks without full payment

Contract

ListaV3Pool.sol

Severity

Low

Description

ListaV3Pool.mint() and UniswapV3Pool.flash() verify that the pool received the required tokens by reading the pool's token balance before calling the user supplied callback and then comparing it against the post callback balance.

If a token automatically accrues balance to holders, for example by exposing a public function that mints accrued interest into the holder's balance, an attacker can trigger that interest payout to the pool from inside the callback. The accrued interest then counts toward the required balance increase, and the attacker can mint liquidity or take a flash loan while paying less than the protocol expects.

Description

JavaScript

```
/// @inheritdoc IListaV3PoolActions
/// @dev noDelegateCall is applied indirectly via _modifyPosition
function mint(
    address recipient,
    int24 tickLower,
    int24 tickUpper,
    uint128 amount,
    bytes calldata data
) external override lock returns (uint256 amount0, uint256 amount1)
{
    require(amount > 0);
    (, int256 amount0Int, int256 amount1Int) =
        _modifyPosition(
            ModifyPositionParams({
                owner: recipient,
```

```

        tickLower: tickLower,
        tickUpper: tickUpper,
        liquidityDelta: int256(amount).toInt128()
    })
};

amount0 = uint256(amount0Int);
amount1 = uint256(amount1Int);

uint256 balance0Before;
uint256 balance1Before;
if (amount0 > 0) balance0Before = balance0();
if (amount1 > 0) balance1Before = balance1();

IUniswapV3MintCallback(msg.sender).uniswapV3MintCallback(amount0,
amount1, data);
if (amount0 > 0) require(balance0Before.add(amount0) <=
balance0(), 'M0');
if (amount1 > 0) require(balance1Before.add(amount1) <=
balance1(), 'M1');

emit Mint(msg.sender, recipient, tickLower, tickUpper, amount,
amount0, amount1);
}

```

Consider documenting clearly that the pool does not support tokens with passive yield or rebasing behavior and warn integrators about reduced effective payments when such tokens are paired.

Recommendation

When stronger guarantees are required, track the explicit transferred amount, for example by recording the difference between an internally tracked accounting balance and the live ERC20 balance, or restrict the protocol to a curated set of tokens that do not exhibit this behavior.

Status

Acknowledged inherited uniswapV3 behavior, no fix implemented. Listadao has indicated that tokens with rebasing behavior are not supported.

[I04] Missing zero address check in setOwner

Contract ListaV3Factory.sol

Severity Informational

Description

The ListaV3Factory.setOwner() function does not validate that the new owner address is non zero. While the initialize() function correctly enforces require(_owner != address(0)), this same check is absent from setOwner(), which was inherited from the original Uniswap implementation.

If the current owner accidentally or maliciously transfers ownership to address(0), all owner gated functionality becomes permanently inaccessible. This includes the ability to enable new fee tiers and configure protocol fees, leaving the factory in a state where its administrative controls cannot be recovered.

JavaScript

```
/// @inheritdoc IListaV3Factory
function setOwner(address _owner) external override {
    require(msg.sender == owner);
    emit OwnerChanged(owner, _owner);
    owner = _owner;
}
```

Recommendation	Consider adding a require(_owner != address(0)) check at the beginning of setOwner() to prevent ownership from being transferred to the zero address.
-----------------------	---

Status	Acknowledged inherited uniswapV3 behavior, no fix implemented. ListaDao has indicated that the owner is a trusted multi-sig and will avoid such input errors.
---------------	---

[I05] Missing validation of _owner argument could lead to incorrect event emission

Contract	ListaV3Factory.sol
-----------------	--------------------

Severity	Informational
-----------------	---------------

Description

Because the setOwner lacks input validation, the owner can be updated to the existing owner. Although such an update wouldn't change the contract state, it would emit an event falsely indicating the owner had been changed.

JavaScript

```
/// @inheritdoc IListaV3Factory
function setOwner(address _owner) external override {
    require(msg.sender == owner);
    emit OwnerChanged(owner, _owner);
    owner = _owner;
}
```

Description	
--------------------	--

Recommendation	Consider including an input validation check that ensures the _owner input is not equal to the current owner
-----------------------	--

Status	Acknowledged inherited uniswapV3 behavior, no fix implemented. ListaDao has indicated that the owner is a trusted multi-sig and will avoid such input errors.
---------------	---

[I06] getNextSqrtPriceFromInput and getNextSqrtPriceFromOutput do not enforce MIN_SQRT_RATIO and MAX_SQRT_RATIO bounds

Contract SqrtPriceMath.sol & TickMath.sol

Severity Informational

Description

SqrtPriceMath.getNextSqrtPriceFromInput and SqrtPriceMath.getNextSqrtPriceFromOutput return a next square root price without verifying that the result lies within the documented [MIN_SQRT_RATIO, MAX_SQRT_RATIO] range.

Currently, an out of range result is caught downstream by the require in TickMath.getTickAtSqrtRatio, so the bug is not directly exploitable. However, the safety of these functions depends entirely on every caller routing the output through that downstream check. A future refactor that removes or bypasses the check, or a new caller that uses the value without converting it back to a tick, would silently accept out of bounds prices and break invariants throughout the system.

The square ratio price's limit is defined with MIN_SQRT_RATIO/MAX_SQRT_RATIO:

JavaScript

```
/// @dev The minimum value that can be returned from
# getSqrtRatioAtTick. Equivalent to getSqrtRatioAtTick(MIN_TICK)
uint160 internal constant MIN_SQRT_RATIO = 4295128739;
/// @dev The maximum value that can be returned from
# getSqrtRatioAtTick. Equivalent to getSqrtRatioAtTick(MAX_TICK)
uint160 internal constant MAX_SQRT_RATIO =
1461446703485210103287273052203988822378723970342;
```

Description

getNextSqrtPriceFromInput/getNextSqrtPriceFromOutput returns a next square price ratio based on the current one:

JavaScript

```
/// @notice Gets the next sqrt price given an input amount of token0
or token1
/// @dev Throws if price or liquidity are 0, or if the next price is
out of bounds
/// @param sqrtPX96 The starting price, i.e., before accounting for
the input amount
/// @param liquidity The amount of usable liquidity
/// @param amountIn How much of token0, or token1, is being swapped
in
/// @param zeroForOne Whether the amount in is token0 or token1
```

```

    /// @return sqrtQX96 The price after adding the input amount to
    token0 or token1
    function getNextSqrtPriceFromInput(
        uint160 sqrtPX96,
        uint128 liquidity,
        uint256 amountIn,
        bool zeroForOne
    ) internal pure returns (uint160 sqrtQX96) {
        require(sqrtPX96 > 0);
        require(liquidity > 0);

        // round to make sure that we don't pass the target price
        return
            zeroForOne
                ? getNextSqrtPriceFromAmount0RoundingUp(sqrtPX96,
liquidity, amountIn, true)
                : getNextSqrtPriceFromAmount1RoundingDown(sqrtPX96,
liquidity, amountIn, true);
    }

    /// @notice Gets the next sqrt price given an output amount of
    token0 or token1
    /// @dev Throws if price or liquidity are 0 or the next price is out
    of bounds
    /// @param sqrtPX96 The starting price before accounting for the
    output amount
    /// @param liquidity The amount of usable liquidity
    /// @param amountOut How much of token0, or token1, is being swapped
    out
    /// @param zeroForOne Whether the amount out is token0 or token1
    /// @return sqrtQX96 The price after removing the output amount of
    token0 or token1
    function getNextSqrtPriceFromOutput(
        uint160 sqrtPX96,
        uint128 liquidity,
        uint256 amountOut,
        bool zeroForOne
    ) internal pure returns (uint160 sqrtQX96) {
        require(sqrtPX96 > 0);
        require(liquidity > 0);

        // round to make sure that we pass the target price
        return
            zeroForOne
                ? getNextSqrtPriceFromAmount1RoundingDown(sqrtPX96,
liquidity, amountOut, false)
                : getNextSqrtPriceFromAmount0RoundingUp(sqrtPX96,
liquidity, amountOut, false);
    }

```

Both functions allow the next square ratio to be outside of its expected bounds. Currently, the issue is not exploitable, as the bound is checked in `getTickAtSqrtRatio`:

JavaScript

```
function getTickAtSqrtRatio(uint160 sqrtPriceX96) internal pure
returns (int24 tick) {
    // second inequality must be < because the price can never reach
    the price at the max tick
    require(sqrtPriceX96 >= MIN_SQRT_RATIO && sqrtPriceX96 <
MAX_SQRT_RATIO, 'R');
```

Recommendation

Consider adding an explicit bounds checks at the end of `getNextSqrtPriceFromInput` and `getNextSqrtPriceFromOutput` that revert when the computed next price is outside `[MIN_SQRT_RATIO, MAX_SQRT_RATIO]`. Document the exact input and output bounds for every arithmetic helper in the math libraries and back the documented bounds with property based tests so that a refactor cannot silently regress them.

Status

Acknowledged inherited uniswapV3 behavior, no fix implemented.

Resources & Appendix

Known Audit Issues

Description	V3-Core
	• ABDK Audit Report • Trail Of Bits Audit Report
	V3-Periphery
	• ABDK Audit Report

Known External Integration Issues for UniswapV3

Description	• TWAP oracle manipulation on low-liquidity / new pools, Euler / Rari-style. Uniswap V3 oracle is manipulation-resistant only when liquidity is deep and the TWAP window is long enough that single-block manipulation is uneconomic. Other References: TWAP Attack Simulator
	• First-depositor / inflation attack on LP-as-collateral wrappers, when the wrapper mints shares 1:1 with deposited liquidity but value is read from slot0, a tiny first deposit + price manipulation lets an attacker dilute later depositors.
	• SwapRouter deadline / slippage, historical issues where integrations passed deadline = type(uint256).max and amountOutMinimum = 0, exposing users to MEV. Lista wrappers must enforce sane defaults.
	• refundETH / stuck WETH, pattern where router holds residual ETH; ensure no Lista flow relies on the router being empty between calls.
	• Callback impersonation any contract that implements uniswapV3SwapCallback / uniswapV3MintCallback / uniswapV3FlashCallback MUST verify msg.sender == computed pool.
	• Long-running pool accumulator overflow, feeGrowthGlobal{0,1}X128, tickCumulative, secondsPerLiquidityCumulativeX128 are designed to wrap. Any Lista consumer that subtracts these between two snapshots must use unchecked semantics (mirror what Oracle.observeSingle does).
	• Periphery contracts inherit Multicall and many functions are payable. A known class of bugs is possible, where a function that uses msg.value inside a multicall will see the same msg.value on every sub-call.

Upgradeability

Description

- ProxyAdmin ownership / timelock. The ProxyAdmin currently transfers to PROXY_ADMIN_OWNER (defaults to OWNER, the same EOA / multisig that owns the factory). A compromised PROXY_ADMIN_OWNER can upgradeTo an arbitrary impl and drain every pool created by the factory. As such, it is recommended that the ListaDao Timelock contract holds the ownership of the ProxyAdmin
- Selector clash with ProxyAdmin. The Transparent upgradeable proxy forwards admin calls (admin, implementation, upgradeTo, upgradeToAndCall, changeAdmin) to itself. None of the implementation contracts (Factory, NPM) define these selectors, so no clash exists currently, however, it is, worth re-checking on every upgrade.
- Storage layout compatibility on upgrades. Any new variable must be appended to the end of the implementation's existing storage (and the relevant __gap[N] shrunk by the same number of slots). Never reorder, remove, or change variable types in a way that changes slot size.
- Immutables on implementations. factory and WETH9 are immutables on the NPM contract, On NPM upgrades, the new impl bytecode must be compiled with the same (factoryProxy, WETH9). Misconfiguring the impl on upgrade can silently change which factory the NPM is bound to.

Considerations for ListaDao

Description

1. Lista Lending (Moolah)

- Most markets currently delegate oracle calls to RESILIENT_ORACLE (e.g. ChainlinkOracle/Redstone/BinanceOracle). If a future adapter pulls a price from a ListaV3 pool or if the resilient oracle integrates a ListaV3Pool as a oracle option, treat it as a TWAP oracle and
 - Require observationCardinality $\geq N$,
 - Implement a minimum TWAP window (≥ 30 min on BNB given 3 s blocks ≈ 600 blocks),
 - Reject pools whose liquidity falls below a configurable floor (to prevent low liquidity manipulation similar to the Venus vThena market attack)

- Cross-check against Chainlink / RESILIENT_ORACLE and revert on > X% divergence
- If a ListaV3 NFT (LP position) is whitelisted as collateral via a wrapper (mirroring PancakeSwapV3LpProvider in lista-dao-contracts, evident in the [CLAUDE.md](#) V3Provider indicated), consider inheriting and considering the following security concerns. Additionally, the new provider contracts must be audited to ensure zero integration errors
 - Conservative LLTV/Collateralization Ratio.
 - Donation attacks via increaseLiquidity from outside the provider (anyone can call NPM increaseLiquidity for a tokenId they don't own, by passing amount0/1Desired and approving). The provider must reject donations either by not exposing the tokenId, by tracking position.liquidity against an internal accounting variable, or by claiming all unclaimed tokensOwed regularly into the share base.
 - Range-reset risk: if the manager can change the position's tickLower/tickUpper, every reset takes a slippage hit and re-exposes the position to pricing changes
 - Pricing must follow a secure pattern, e.g. GUni LP pattern that is already used in PcsV3LpNumbersHelper.computeFairSqrtPriceX96, where the sqrtPriceX96 is derived from the RESILIENT_ORACLE quotes, then plug into LiquidityAmounts.getAmountsForLiquidity with tickLower/tickUpper from INonfungiblePositionManager.positions(). The slot0().sqrtPriceX96 is directly never met for valuation.
 - Liquidation helper via provider (e.g. perform liquidity removal to underlying tokens), Moolah.liquidate calls IProvider(provider).liquidate(id, borrower) after transferring seized collateral to the liquidator within the Moolah contract. If the liquidator is unable to redeem the share back into the underlying NFT (e.g. pool is locked, one of the two tokens is paused), the liquidation completes on the books but the liquidator is stuck with un-redeemable shares.

- In the future V3Provider contract, it must use the exact NPM address and implement the necessary onERC721Received to allow collateral transfers. If not, an equivalent collateral token must be minted to the user based on the value of the ListaV3 LP token
- Reentrancy from inside a swap callback is not possible. Although the V3 swap callback runs while the pool is locked for the pool, nothing prevents the callback from calling back into Moolah / SlisBNBxMinter. Add the nonReentrant guard everywhere the provider exposes a state-mutating entrypoint.
- For SlisBNBxMinter registration, if V3Provider registers as a module in SlisBNBxMinter (analogous to SmartProvider), it must implement an equivalent getUserBalanceInBnb (liquidity in BNB-equivalent units).
- Several contracts such as LendingBroker and Liquidator (Liquidator, BrokerLiquidator and PublicLiquidator) contracts uses 1inch via a router whitelist and routerWhitelist toggle. If a ListaV3 swap path is added the broker must:
 - Validate recipient and deadline,
 - Enforce user inputted slippages
 - Account for fee-on-transfer mismatches between expected and actual amountOut if allowed.

2. Lista CDP

- Here, PancakeSwapV3LpProvider.sol, serves as an equivalent similar contract if a ListaV3 LP token is ever whitelisted for use as a collateral for a lisUSD CDP position.
- PcsV3LpNumbersHelper.sol prices a position by reading slot0 and computing token amounts from the position's tick range. Ensure that:
 - TWAP fallback from the same pool with min cardinality,
 - oracle cross-check (resilientOracle) and a max deviation,
 - explicit handling of in-range vs out-of-range positions (out-of-range = single-sided exposure; valuation must use the boundary tick price, not slot0).

- CDP liquidation path via CDPLiquidator.sol, clip.sol, dog.sol, FlashBuy.sol.
 - Ensure the liquidator can actually unwind a ListaV3 LP under stress
 - decreaseLiquidity requires $\text{block.timestamp} \leq \text{deadline}$ and the pool to be unlocked.
 - A reentrant token in the pair would block liquidation.
- PSM: if a ListaV3 pool is ever paired against lisUSD, peg arbitrage will pull from it. Ensure that low-liquidity or single-provider pools is not used in any peg-sensitive code path.

3. Lista Token (BuyBacks)

- BuyBack routes via 1inch (no direct V3 path currently). If a direct ListaV3 swap path is added, apply
 - Slippages and deadline
 - Possible per-tx volume cap relative to pool liquidity to mitigate sandwich risk, given LISTA is a relatively low-liquidity token at launch, so MEV risk is possible
- VeListaDistributor.sol, VeListaRewardsCourier/V2.sol, if rewards ever pay out via a ListaV3 swap, the recipient should be a vault, not the user, to avoid griefing via callback reentry.